

Práctica 1. Ecuaciones de Bellman, Policy y Value Iteration

(Duración: 2 sesiones)

PREPARACIÓN DEL ENTORNO DE PROGRAMACIÓN

La práctica se ejecuta con una instalación local de Python. No hace falta Docker, ni WSL, ni ningún servidor X11: basta con un intérprete y tres librerías.

1. **Instalar Python 3.10 o superior.** Si no lo tienes, descárgalo de la [página oficial de Python](#) o utiliza una instalación de Anaconda.
2. **Crear un entorno virtual** para la asignatura y activarlo. Desde la carpeta de la práctica:

```
python -m venv .venv
.venv\Scripts\activate      (Windows)
source .venv/bin/activate   (macOS y Linux)
```

3. **Instalar las dependencias** (numpy, gymnasium y pygame):

```
pip install -r requirements.txt
```

4. **Ejecutar la aplicación (en clase):** prueba a ejecutar la siguiente instrucción desde el terminal para comprobar que no hay errores y que puedes visualizar correctamente el entorno (debería abrirse una ventana con el entorno de la Figura 1).

```
python main.py --env_behavior deterministic --algorithm policy_iteration
```

Cualquier editor sirve para trabajar. Si usas VSCode, basta con instalar la extensión de Python y seleccionar el intérprete del entorno virtual que acabas de crear.

1. Evaluación de la práctica

La primera práctica del laboratorio de aprendizaje por refuerzo se divide en dos partes. La primera consiste en un trabajo previo que los alumnos deben subir a la entrega que habrá habilitada en Moodle antes del inicio de la sesión presencial del laboratorio. Para ello se dispone de una semana.

La segunda parte consiste en la implementación y el análisis de los algoritmos Policy y Value Iteration en un entorno tabular. El entregable correspondiente a esta parte de la práctica consiste en un informe en el que se responderán las preguntas que haya a lo largo del enunciado y en el que se deberá mostrar que se ha comprendido los aspectos más importantes del aprendizaje de los agentes. Por ello se valorará que las respuestas estén debidamente justificadas.

El plazo para entregar el informe es de una semana desde la realización de la sesión de laboratorio.

La evaluación final de la práctica ponderará tanto el trabajo previo como el informe, siendo obligatorio haber entregado los previos antes del laboratorio para que se tenga en cuenta la nota del informe.

2. Descripción del problema

Basado en juegos como el Flappy Bird se ha desarrollado un “grid world” en el que un jugador debe moverse desde una celda inicial dada hasta una celda final conocida obteniendo en el camino la mayor recompensa posible. La Figura 1 muestra el entorno con el que vamos a interactuar.

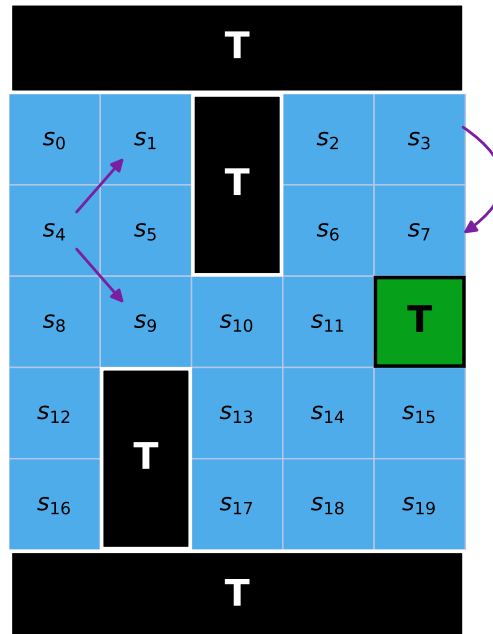


Figura 1. Entorno de la práctica. Solo llevan número las celdas en las que el agente puede encontrarse: son los 20 estados no terminales, s_0 a s_{19} . Todas las celdas marcadas con T son el mismo estado terminal, al que se llega tanto al chocar con un obstáculo (celdas negras) como al alcanzar el objetivo (celda verde). Las flechas ilustran las dos acciones desde s_4 y la caída del borde derecho.

ENTORNO

- Las celdas forman una matriz 7x5. Las celdas azules son los estados en los que el jugador puede encontrarse y se numeran de izquierda a derecha y de arriba a abajo, de s_0 a s_{19} (ver Figura 1).
- Si el agente se mueve desde uno de estos estados a otro estado no terminal, obtiene una recompensa r_s .
- Las celdas negras son obstáculos que el agente ha de evitar, ya que si cae en uno, se recibe una recompensa r_{obs} y se termina el episodio.
- La celda verde representa el objetivo al que hay que llegar, el cual implica una recompensa r_{goal} y también termina el episodio.
- Las celdas negras y la celda verde **no son estados distintos entre sí**: todas ellas son el mismo estado terminal T , en el que el episodio acaba. Lo que las diferencia es la recompensa de la transición que lleva hasta él, no el estado de destino. T es absorbente.

MOVIMIENTOS DEL JUGADOR

- El jugador tiene dos posibles acciones: arriba-derecha (del estado s_4 se mueve al s_1) o abajo-derecha (del estado s_4 se mueve al s_9).
- El entorno tiene bordes sólo por la derecha. Si desde un estado de la última columna se realiza cualquiera de las dos acciones, se cae en la misma columna pero una celda por debajo (del estado s_3 se pasa al s_7 se tome la acción que se tome). Ver Figura 1.
- Si no se indica lo contrario, el entorno y la política son deterministas.

ASUNCIONES (si no se especifica otra cosa)

- Factor de descuento $\gamma = 0,9$
- $r_{obs} = -5$
- $r_{goal} = +5$
- $s_{init} = \{s_0, s_4, s_8, s_{12}, s_{16}\}$, es decir, la primera columna
- T : único estado terminal, absorbente

3. Trabajo previo (2 ptos)

- Determinar, en función de r_s y suponiendo la política estocástica equiprobable en todos los estados, las ecuaciones de Bellman de la función de valor de los estados s_6, s_{10} y s_5 . Se recomienda resolverlas en ese orden.
- Con los valores dados de $r_{obs} = -5$ y $r_{goal} = +5$, y tomando **solo para este apartado** un factor de descuento $\gamma = 1$, determinar el **rango de valores de r_s** para el que la política óptima lleva al agente a la celda objetivo lo antes posible desde todo estado desde el que el objetivo sea alcanzable. Justificar los dos extremos del rango: qué hace el agente cuando r_s queda por encima del límite superior y qué hace cuando queda por debajo del inferior.
- ¿Existe una única política óptima en este entorno? Justificar la respuesta.

4. Policy Iteration (3.5 ptos)

Para los siguientes apartados asumir que $r_s = -1$.

A continuación se debe implementar y analizar el algoritmo de Policy Iteration. Para ello se dispone de una carpeta en la que se encontrarán los siguientes archivos:

- `main.py`: es el script principal que permite ejecutar los algoritmos que se desarrollarán en la práctica. Incluye algunos hiperparámetros importantes como el factor de descuento γ y la tolerancia (`convergence_tolerance`) para la convergencia de los algoritmos, que pueden fijarse también desde la línea de comandos con `--gamma` y `--tol`. Para ejecutar los algoritmos:

```
python main.py --env_behavior <deterministic/stochastic> --algorithm <policy_iteration/value_iteration/both>
```

La opción `--no_render` desactiva la ventana de visualización, y `python main.py --help` lista el resto de opciones.

- `env.py`: contiene la definición del entorno donde el agente interactúa y toma decisiones basadas en las políticas. Está diseñado como un entorno de Gymnasium. Proporciona el modelo de transiciones P , el número de estados nS (21: los 20 visitables y el terminal T) y el número de acciones nA . No es necesario modificar nada dentro de este código durante la práctica.
- `policy_iteration.py`: implementación base para el algoritmo. Dentro se encontrarán un método para desarrollar Policy Evaluation y otro para Policy Improvement.

Programar dentro del espacio correspondiente (marcado con `START CODE HERE ... END CODE HERE`) los métodos de `policy_iteration.py` para obtener $V_*(s)$ y π_* para todos los estados. Primero se programará la parte de la evaluación de la política, luego la mejora de la política y por último la iteración. No se debe modificar nada que no esté dentro de este espacio, los argumentos que recibe el método son suficientes para desarrollar el algoritmo.

Analizar el proceso de aprendizaje del agente y realizar un análisis sobre cómo se ve afectado el aprendizaje del agente y la convergencia del mismo para distintos valores de los hiperparámetros γ y umbral de convergencia θ .

5. Value Iteration (3.5 ptos)

A continuación se deben obtener $V_*(s)$ y π_* para todos los estados mediante Value Iteration. Para ello se dispone en la misma carpeta descrita anteriormente el archivo `value_iteration.py` que debe cumplimentarse del mismo modo que en el apartado previo. No se debe modificar nada que no esté dentro de este espacio, los argumentos que recibe el método son suficientes para desarrollar el algoritmo.

Analizar el proceso de aprendizaje del agente y realizar un análisis sobre cómo se ve afectado el aprendizaje del agente y la convergencia del mismo para distintos valores de los hiperparámetros γ y umbral de convergencia θ .

Comparar el desempeño del agente cuando aprende mediante Policy Iteration y cuando lo hace con Value Iteration. ¿Se llega siempre a la misma solución?, ¿son las políticas óptimas iguales?, ¿convergen en tiempos similares?

6. MDP estocástico (1 pto)

Asumir ahora que las transiciones del entorno pasan a ser estocásticas, de manera que: en todos los estados y pares (estado, arriba-derecha) el agente realiza la acción arriba-derecha con $p_{\text{exito}} = 0,8$ y con $p_{\text{fallo}} = 0,2$ va abajo-derecha. Del mismo modo, para todos los estados y pares (estado, abajo-derecha) el agente realiza la acción abajo-derecha con $p_{\text{exito}} = 0,8$ y con $p_{\text{fallo}} = 0,2$ va arriba-derecha. Analizar el proceso de aprendizaje del agente y compararlo con su desempeño en el entorno determinista.